

Data Description - Instances and Results

Carolin Hasse and Stefan Irnich

June 9, 2026

1 General

This is a description of the instance and fleet files used and the results reported in the working paper *Branch-price-and-cut for the vehicle routing problem with simultaneous delivery and pickup, time windows, and load-dependent cost*, see <https://logistik.bwl.uni-mainz.de/tech-reports/>. The files are available at <https://github.com/carolinhasse/VRPSDPTW-LDC>.

The studies performed in the paper are hereafter denoted by a number x , where x indicates the order in which the study appears in the paper. The CSV files reporting our results are named `study $x$ _results.csv`, accordingly. For clarity, the mapping between studies and their descriptions is as follows:

- Study 1: Impact of the load-dependent BPC extension on computational performance.
- Study 2: Effect of the instance size on the performance of the BPC algorithm.
- Study 3: Effect of the load-impact factor ρ in the linear continuous load-dependent cost function $c_{ij}(q_{ij}) = c_{ij} \cdot (1 + q_{ij}/Q \cdot \rho)$.
- Study 4: Impact of discontinuity location in the load-dependent cost functions.
- Study 5: Effects of an increasing load-impact factor ρ on four solution metrics.
- Study 6: Comparison of fleets $\text{fleet}_{\text{het}}$ and $\text{fleet}_{\text{lift}}$.

2 Instance Description

2.1 Instances

For the Studies 1, 3, 4 and 6, the instances used are the VRPSDPTW benchmark instances with 100 customers of Solomon (1987), adapted by Wang and Chen (2012), available at <https://oz.nthu.edu.tw/~d933810/test.htm>. The 56 instances are classified by customer distribution as random (R), clustered (C), and mixed (RC). The instances can also be divided into Series 1 with the subsets R1, C1, and RC1 with tight constraints and Series 2 with the subsets R2, C2 and RC2 with loose constraints. The latter are on average harder to solve.

Study 2 uses instances with random subsets of 20, 40, 60, and 80 customers of the VRPSDPTW instances described above. We name these instance sets VRPSDPTW.20, VRPSDPTW.40, VRPSDPTW.60, and VRPSDPTW.80 and provide them at this repository.

Study 5 uses the instance RDP101 of the VRPSDPTW benchmark instances exemplarily to showcase the effects of the load-impact factor ρ . In its original version, the instance has 100 customers and uses a non-restrictive fleet of 25 homogeneous vehicles with a capacity of $Q = 200$ each. For this study, we use the original instance RDP101 and two modified instances RDP101.Q50 and RDP101.Q50.F31. For these modified instances, the capacity is reduced to $Q = 50$, which makes the previous fleet of 25 vehicles restrictive. We therefore set the fleet to be unlimited (or rather to the number of customers, i.e., 100) for RDP101.Q50 and to at most 31 vehicles for RDP101.Q50.F31. These two modified instances can also be downloaded from this repository.

2.2 Fleets

The BPC algorithm described in our paper is able to handle heterogeneous fleets where the vehicles may have differing cost functions and/or differing capacities. We define the cost functions and capacities in fleet files that use the VRPSDPTW instance files with customer data as a kind of basis. The instances already define a vehicle capacity. In the fleet files, we therefore only define (non-negative) capacity factors.

The arc routing costs follow a similar reasoning. Recall the triplets (Q_s, m_s, δ_s) used to describe the cost functions as explained in Section 3.2 of our paper. In the fleet files, we use related triplets (x_s, m_s, ρ_s) , i.e., the triplets use a (non-negative) “x-value factor” x_s rather than the (integer) breakpoint Q_s and the load-impact factor ρ_s rather than the slope δ_s . The relationship between the triplets defined in the paper and the triplets in the fleet files is given by

$$\begin{aligned} Q_s &= x_s \cdot Q, \\ m_s &= m_s \text{ and} \\ \delta_s &= \rho_s / Q. \end{aligned}$$

Format: First, the number of vehicle types is given. Each vehicle type is then specified by its capacity factor, the number of segments in its cost function, and the corresponding sequence of segment triplets (x_s, m_s, ρ_s) . This structure is repeated for every vehicle type contained in the fleet file.

```

1  (int)    % number of vehicle types
2
3
4  (float) % capacity factor for vehicle type 1
5  (int)    % number of pieces in the cost function of vehicle type 1
6
7  (float) % x-value factor of first triplet of cost function of vehicle type 1
8  (float) % base cost multiplier of first triplet of cost function of vehicle type 1
9  (float) % load-impact-factor of cost function of vehicle type 1, convert to slope
10
11
12 (float) % capacity factor for vehicle type 2
13 (int)    % number of pieces in the cost function of vehicle type 2
14
15 (float) % x-value factor of first triplet of cost function of vehicle type 2
16 (float) % base cost multiplier of first triplet of cost function of vehicle type 2
17 (float) % load-impact-factor of first triplet, convert to slope
18
19 (float) % x-value factor of second triplet of cost function of vehicle type 2
20 (float) % base cost multiplier of second triplet of cost function of vehicle type 2
21 (float) % load-impact-factor of second triplet, convert to slope

```

Listing 1: Format of fleet files.

Example: Consider a heterogeneous fleet with two types of vehicles $k = 1, 2$. Vehicles of type $k = 1$ may use 100% of the capacity stated in the instance file, i.e., $Q^1 = Q$. The vehicles of type $k = 2$ have a capacity of 110% of the capacity stated in the instance file, i.e., $Q^2 = 1.1 \cdot Q$. Let the first vehicle type have load-independent routing costs, i.e., let the arc costs be equal to the Euclidean distance between two nodes and not rise in load. Let the second vehicle type have a piecewise linear cost function with two segments (see also Figure 1).

$$c_{ij}^1(q_{ij}) = \underline{c}_{ij} \cdot (1.0 + q_{ij}/Q \cdot 0.0) = \underline{c}_{ij}, \quad q_{ij} \in [0, Q^1 = Q]$$

$$c_{ij}^2(q_{ij}) = \begin{cases} \underline{c}_{ij} \cdot (1.00 + q_{ij}/Q \cdot 0.3), & \text{if } q_{ij} < 0.6 \cdot Q \\ \underline{c}_{ij} \cdot (1.01 + q_{ij}/Q \cdot 0.4), & \text{if } q_{ij} \geq 0.6 \cdot Q \end{cases}$$

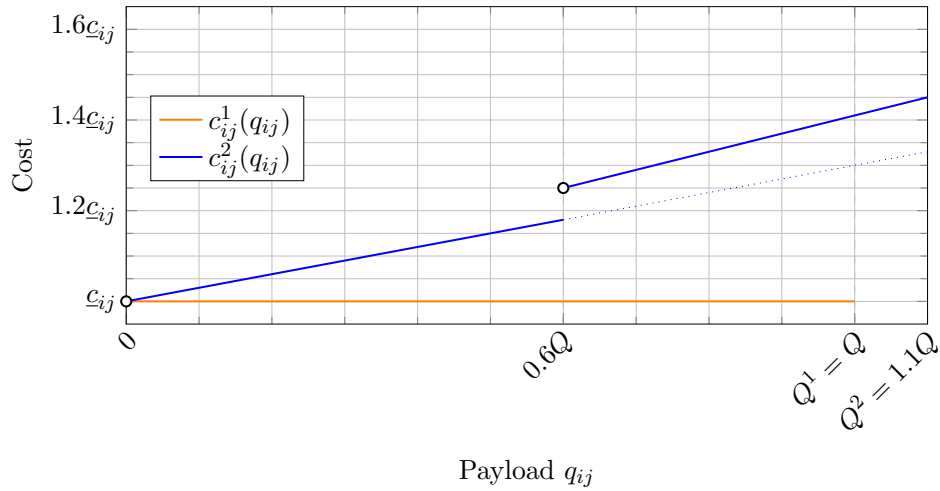


Figure 1: Graphs of the cost functions in the example.

For this fleet, the file would look like so:

1	2
2	1
3	0.0
4	1.0
5	0.0
6	1.1
7	2
8	0.0
9	1.0
10	0.3
11	0.6
12	1.25
13	0.4

Listing 2: Example fleet file with two vehicle types.

3 Results

Table 1 summarizes the variables available across the six studies. Because the study datasets differ slightly in content, some variables are available only for a subset of studies.

Cell title	Meaning
instance name	self-explanatory
fleet name	self-explanatory
opt?	binary value, true when solved to optimality within given time limit
UB	best known upper bound, 1.00E+11 when no upper bound is found
LB tree	best known lower bound, -2.50E+13 when no lower bound is found
LB root	lower bound at root node, -2.50E+13 when no lower bound is found
LB root+cuts	lower bound after cuts at root node, -1.00E+9 when no lower bound is found
time [s]	solution time in seconds, 3600 if not solved to optimality within time limit
time root [s]	solution time of root node, 3600 if not solved within time limit
time root+cuts [s]	solution time after cuts at root node, 3600 if not solved within time limit
time limit [s]	self-explanatory
BB nodes	number of branch-and-bound nodes solved until optimal solution is found or time limit is reached
routes	best known solution, 0 if none is found
RN solved?	binary value, true when root node is solved within given time limit
Pieces RN	zero-based vector where entry i gives the number of labels whose reduced cost function has exactly i segments for all labels generated in the root node
Pieces BPC	zero-based vector where entry i gives the number of labels whose reduced cost function has exactly i segments for all labels generated

Table 1: Data reported in the studies.

References

- Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265.
- Wang, H.-F. and Chen, Y.-Y. (2012). A genetic algorithm for the simultaneous delivery and pickup problems with time window. *Computers & Industrial Engineering*, 62(1):84–95.